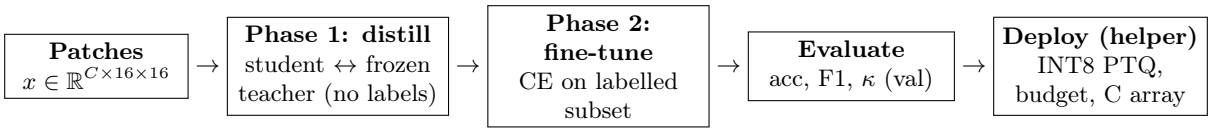
tinygfm: Research Overview

Tiny Multi-Task Network for On-Board AI Inference in Earth Observation, distilled from Geospatial Foundation Models

Davide Galluzzo — review handout

1 Scope and status

`tinygfm` is a PyTorch scaffold that tests one question: *at equal model size (≤100 KB of INT8 weights), when does a student CNN distilled from a frozen Geospatial Foundation Model (GFM) beat the same student trained conventionally?* Factors varied: student size, label fraction, spectral bands, baseline type, single vs. multi-task head.



Pipeline: distil, fine-tune, evaluate, deploy.

2 Dataset

Training has two phases. In Phase 1 the student imitates the teacher on unlabelled Sentinel-2 patches, so it can use a large and diverse pool; in Phase 2 the heads are fine-tuned on the labelled patches of CloudSEN12, Sen1Floods11 and THRawS.

2.1 SSL4EO-S12 (unlabelled Sentinel-2 pool for distillation)

It has 251,079 locations, mostly around large cities, each seen in four seasons, with images of 264 × 264 px (2.64 km at 10 m) and Sentinel-2 L1C (13 bands). That gives about 1 M images, or 2.6×10^8 patches of 16×16[†]; a random subset of a few hundred thousand images across seasons and locations is enough.

Points to check.

- *Few clouds.* Only scenes with under 10% cloud cover are kept, so the unlabelled CloudSEN12 images are added to the pool for the cloud task.
- *Disjoint areas.* Pool images that overlap the validation or test areas of the three datasets are removed.
- *Teacher overlap.* Several foundation models were pre-trained on SSL4EO-S12, so teacher and student may see the same images; this is declared.

2.2 CloudSEN12 (cloud and cloud shadow, control task)

Content. Global Sentinel-2 scenes (2018–2020). Each region of interest (ROI) is 5,090 × 5,090 m at 10 m, so one image patch (IP) is ≈ 509 × 509 px, and each ROI has several IPs with different cloud cover. High-quality labels: 2,000 ROIs = 10,000 IPs; no label: ≈29,250 IPs. Classes: clear, thick cloud, thin cloud, cloud shadow. Data: Sentinel-2 L1C (13 bands) and L2A, plus Sentinel-1 and DEM.

Labels and split. 14 trained experts, with human agreement of 95.7% (thin cloud is the weak spot, about 78%); a U-Net with MobileNetV2 reaches 0.92 balanced accuracy, versus 0.84 for Fmask. Spatial hexagon split: 1,827 training and 173 test ROIs; all IPs of an ROI stay in the same split.

2.3 Sen1Floods11 (surface water)

Content. 11 flood events, 4,831 chips of 512 × 512 px at 10 m. Sentinel-2 L1C (13 bands, UInt16, reflectance ×10,000) and Sentinel-1 (VV, VH); labels −1 no data, 0 not water, 1 water.

Labels and split. 446 *hand-labelled* chips, 4,385 *weakly labelled* chips (automatic rules, no quality control) and 814 JRC chips. Hand-labelled split: 252 train, 89 validation, 90 test, plus the Bolivia event held out as a geographic test (15 chips[†]).

2.4 THRawS (thermal hotspots in raw Sentinel-2)

Content. Raw (Level-0) Sentinel-2 granules built for on-board processing: only B8A, B11, B12 at 20 m; 88 events (58 volcanic, 20 wildfires, 10 hard negatives).

Labels. Bounding boxes, not masks, produced by a simplified Massimetti et al. algorithm and checked by two experts. Patches of 128 × 128 px with 25% overlap: 1,090 hotspot vs. 33,335 event-free (3.17% positive). No official split and no benchmark results.

2.5 Patch counts and memory at 16×16

Cutting each image into non-overlapping 16×16 patches gives the counts below. One patch takes $4\,C \cdot 256$ bytes in float32 (13,312 B with 13 bands, 3,072 B with 3 bands), and half of that as UInt16.

Subset	images	patches†	float32†	UInt16†
Sen1Floods11, hand-labelled	446	456,704	6.1 GB	3.0 GB
Sen1Floods11, all chips (weak labels)	4,831	4,946,944	65.9 GB	32.9 GB
CloudSEN12, high-quality	10,000	9,610,000	127.9 GB	64.0 GB
CloudSEN12, no label	29,250	28,109,250	374 GB	187 GB
THRawS (C=3)	34,425	≤2,203,200	6.8 GB	3.4 GB

Patch counts and storage per dataset (derived estimate).

The totals are large, so a subsample of about 50–100 k patches per dataset and split is enough for the experiments, and the unlabelled CloudSEN12 images are not needed in full. THRawS counts are an upper bound, since its patches overlap by 25%. Labelled fractions are always drawn by whole chip, image or event, never by single patch, because neighbouring patches look almost identical and would leak labels between training and test.

2.6 Why these three datasets, and what each one tests

The thesis trains one tiny network on three on-board decisions that share the same input type but differ in physics and class balance.

	CloudSEN12	**Sen1Floods11**	**THRawS**
Patch label	cloud / not cloud	water / not water	hotspot / none
Sensor data	Sentinel-2, 10 m	Sentinel-2, 10 m	raw Sentinel-2, 20 m
Bands	13	13	3 (B8A, B11, B12)
Ground truth	pixel masks	pixel masks	bounding boxes
Positive patches	moderate	low–moderate	very rare (3.17%)
Role	control task	medium difficulty	hardest task

The three datasets compared.

2.7 From masks and boxes to patch labels

The student sees a 16×16 patch P and outputs one class. CloudSEN12 and Sen1Floods11 give a label for every pixel; THRawS gives bounding boxes, which we turn into a pixel mask ($m(p) = 1$ inside a box, 0 outside). The patch label is then computed from the mask of its 256 pixels.

Cloud and water. Let $m(p) = 1$ if pixel p belongs to the target class (cloud = thick or thin; water) and 0 otherwise, ignoring invalid pixels (e.g. label -1). With V the set of valid pixels in P, we compute the valid fraction v and the covered fraction ρ:

$$v = \frac{|V|}{256}, \qquad \rho = \frac{1}{|V|} \sum_{p \in V} m(p). \tag{1}$$

The patch is discarded if $v < v_{\min}$; otherwise

$$y = \begin{cases} 1 & \rho \geq \tau \\ 0 & \rho < \tau. \end{cases} \tag{2}$$

The thresholds τ and $v_{\min}$ are fixed once and reported; a higher τ gives fewer but cleaner positives, and we report results for $\tau \in \{0.05, 0.25, 0.5\}$.

Hotspots. A hotspot can cover only a few pixels, so we count them instead of using a fraction: the patch is positive if

$$\sum_{p \in P} m(p) \geq n_{\min}, \tag{3}$$

and negative otherwise. A box is larger than the burning area, so these labels are coarser than the pixel masks; $n_{\min}$ is chosen on the validation set. At 20 m a patch covers 320 m (160 m at 10 m for the other datasets), so THRawS is reported separately. Without overlap, an $H \times W$ image gives $N = \lfloor H/16 \rfloor \lfloor W/16 \rfloor$ patches.

2.8 Splits and leakage: evaluation protocol

Neighbouring 16×16 patches are almost the same image, so a split made patch by patch lets the test set "see" the training set and inflates every result. The protocol below is proposed for the thesis; only the official splits come from the dataset sources.

Split by group. All patches of the same group (ROI, chip or event) always go to the same split, so training, validation and test share no group. Official splits are used where they exist.

Dataset	Group	Split
CloudSEN12	ROI	Official: 1,827 train / 173 test ROIs. Validation taken from train, by ROI.
Sen1Floods11	Event	Official: 252 train / 89 val / 90 test chips.
THRawS	Event	None official. Own split by event, with volcanoes, wildfires and non-events in every split.

Split rules for each dataset.

Labels, distillation and statistics. Smaller label fractions (e.g. 1, 5, 10, 100%) are nested subsets of whole groups, identical for the distilled model and for every baseline at a given seed. The unlabelled pool for Phase 1 never includes validation or test images. Band statistics are computed on training data only, the decision threshold is set on validation, and the test set is evaluated once.

Metrics. Accuracy is misleading when positives are rare (at 3.17% positives, predicting always "no hotspot" scores 96.8%), so we report the true positive rate at a fixed false positive rate (set on validation), plus F1 and precision–recall AUC. For each seed and label fraction we compare distilled and baseline models on the same test set, $\Delta = M^{\text{distill}} - M^{\text{baseline}}$, and estimate its 95% interval by resampling test *groups*, not patches, 1,000 times. A result counts as an improvement only if the interval excludes 0.

Leakage checks. (a) no group appears in two splits and no duplicate patch crosses splits; (b) with shuffled training labels the test performance must fall to chance; (c) one experiment is repeated with a random patch-level split, and the gap to the group-level result shows how optimistic a naive split would be. The foundation-model teachers may have seen the test areas during pre-training; this cannot be removed, so it is declared and applies equally to distilled models and baselines.

2.9 Data preparation and teacher targets

Using CloudSEN12, Sen1Floods11 and THRawS requires three choices, described below.

1. Data preparation. Each image is cut into 16×16 patches. For every patch we keep its group (ROI, chip or event), its position, the valid fraction v and the covered fraction ρ (or the hotspot pixel count), so that the labelling thresholds τ and $n_{\min}$ can be changed without cutting the images again. Splits and label fractions follow the group-level protocol of Section 2.8.

2. Bands and normalisation. Bands are selected by name, not by position. THRawS has only B8A, B11 and B12, whereas the teacher expects more bands (Prithvi-EO-2.0 uses six: blue, green, red, narrow NIR, SWIR1, SWIR2), so the missing ones are either filled with zeros, and this is reported, or the teacher is given a reduced view. Sentinel-2 L1C values are reflectance scaled by 10,000, while THRawS raw values are not reflectance, so each band is normalised with statistics computed on training data only.

3 Teacher models

A geospatial foundation model (GFM) is a large network pre-trained without labels on millions of satellite images. In this project the GFM is only the teacher: it runs on the ground during training, produces one feature vector per 16×16 patch, and the tiny student learns to reproduce these features. The GFM is never sent to the satellite. Two candidates are compared: Prithvi-EO-2.0 (NASA and IBM) and Copernicus-FM (Wang et al., 2025).

3.1 Candidate models

Prithvi-EO-2.0 (Szwarcman et al., 2024) is a Vision Transformer (ViT) pre-trained as a masked autoencoder: random patches are hidden and the network learns to reconstruct them. It was trained on 4.2 M samples of the Harmonized Landsat–Sentinel-2 (HLS) archive at 30 m (2014–2023), each a 224×224 crop with up to four dates, and always takes the same six bands: blue, green, red, narrow NIR, SWIR1 and SWIR2.

Copernicus-FM (Wang et al., 2025) is a ViT-Base pre-trained on Copernicus-Pretrain (Sentinel-1, -2, -3, -5P and DEM). Its patch-embedding layer extends the DOFA hypernetwork (Xiong et al., 2024): a small network generates the embedding weights from the central wavelength of each band, so any combination of bands is accepted without retraining. Sentinel-2 inputs are 264×264 px at 10 m with 16×16 patches.

	Prithvi-EO-2.0	**Copernicus-FM**
Architecture	ViT-L (300 M) or ViT-H (600 M)	ViT-Base (≈86 M)
Layers / embedding dim d	24 / 1,024 (300 M)	12 / 768
Bands	6, fixed	any
Resolution	30 m	10 m
Patch size	16 (300 M), 14 (600 M)	16
Input size	224×224, up to 4 dates	264×264

The two candidate teachers.

3.2 Compatibility with the pipeline

The pipeline requires 16×16 tokens at 10 m, 13 bands for cloud and water, 3 bands (B8A, B11, B12) for hotspots, and a low inference cost on a ground GPU. Copernicus-FM meets all requirements natively. Prithvi-EO-2.0 meets them only in part: the 600 M variant uses 14×14 patches, the fixed six bands drop seven of the thirteen Sentinel-2 bands, three of its six bands are missing on THRawS, and it was trained at 30 m.

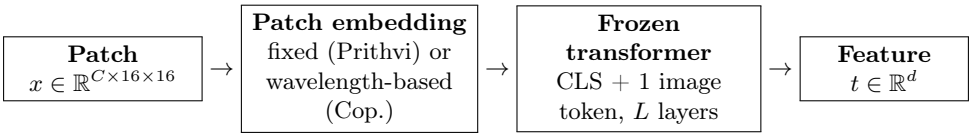
Requirement	**Prithvi-EO-2.0**	**Copernicus-FM**
One token per 16×16 patch	only 300 M	yes
All 13 bands	no (6 of 13)	yes
Only 3 bands (THRawS)	no (3 of 6 missing)	yes
10 m resolution	no (30 m)	yes

Fit of each teacher with the pipeline.

Copernicus-FM is therefore the primary teacher, about 3.6× cheaper per patch, and Prithvi-EO-2.0-300M is kept as a secondary condition.

3.3 Feature extraction and band adaptation

The frozen teacher turns each patch into the distillation target. A patch $x \in \mathbb{R}^{C \times 16 \times 16}$ becomes one image token; a class token (CLS) is prepended, the two-token sequence passes through the frozen transformer, and the CLS output is the feature $t \in \mathbb{R}^d$ (d=768 for Copernicus-FM, 1,024 for Prithvi-300M). With a single image token, CLS and mean-pooling carry the same information. Prithvi is run with one date (temporal index zero), a declared deviation from its multi-temporal pre-training.



Teacher forward pass for one patch; no gradient flows through the teacher.

Bands. Copernicus-FM receives exactly the available bands. Prithvi receives its six bands selected by name; on THRawS blue, green and red are missing and are zero-filled by default, with a band-subset variant as ablation.

	Cloud (CloudSEN12)	**Water (Sen1Floods11)**	**Hotspot (THRawS)**
Input bands C	13	13	3
To Prithvi	B02, B03, B04, B8A, B11, B12	B02, B03, B04, B8A, B11, B12	B8A, B11, B12
To Copernicus-FM	all 13	all 13	B8A, B11, B12

Bands passed to each teacher per task.

Normalisation. Teacher inputs are normalised with the published pre-training statistics of each model, student inputs with training-split statistics (Section 2.9); the two are kept separate and declared.

3.4 Distillation objective

Training has two phases with different goals: in Phase 1 the student learns *how to describe* a patch by imitating the teacher, without labels; in Phase 2 it learns *what to decide* for each task, using a small labelled set. This

subsection defines the quantities and losses of both phases and the size constraint that the final model must respect.

Notation. For a patch $x \in \mathbb{R}^{C\times 16\times 16}$, the frozen teacher T produces the feature $t = T(x) \in \mathbb{R}^d$ (d=768 for Copernicus-FM, d=1,024 for Prithvi-300M), and the student encoder f_θ, with trainable weights θ, produces a much shorter vector $s = f_\theta(x) \in \mathbb{R}^b$, with $b \ll d$ (for example b=64). Because s and t have different lengths, they cannot be compared directly.

Phase 1: matching the teacher. A linear projection head $g_\phi : \mathbb{R}^b \to \mathbb{R}^d$ maps the student vector into the teacher's space, $z = g_\phi(s)$, and the student is trained so that z is as close as possible to t. Two distances are compared:

$$\mathcal{L}_{\text{MSE}} = \frac{1}{d}\|z - t\|^2, \qquad \mathcal{L}_{\cos} = 1 - \frac{z \cdot t}{\|z\|\,\|t\|}. \tag{4}$$

The mean squared error requires z and t to match in both direction and length. The cosine loss only requires them to point in the same direction, so it ignores differences of scale; this makes it more robust when teacher and student inputs are normalised with different statistics. Both losses are run, and the choice is reported for every experiment. No labels are used in this phase, so it can exploit the large unlabelled pool.

Phase 2: learning the tasks. After Phase 1 the projection head g_ϕ is discarded: it was only needed to compare the student with the teacher. On top of the encoder, one small linear head $h_k : \mathbb{R}^b \to \mathbb{R}^2$ is added for each task $k \in \{\text{cloud, water, hotspot}\}$, and gives the two class scores of that task. Each head is trained with the cross-entropy on a labelled batch $\mathcal{B}_k$ of its own dataset,

$$\mathcal{L}_{\text{CE}}^{(k)} = -\frac{1}{|\mathcal{B}_k|} \sum_{(x,y)\in\mathcal{B}_k} \log \hat{p}_y\big(h_k(f_\theta(x))\big), \tag{5}$$

where $\hat{p}_y$ is the predicted probability of the true class $y \in \{0, 1\}$. The three tasks are trained together by summing their losses:

$$\mathcal{L}_{\text{mt}} = \sum_{k=1}^{3} \lambda_k \, \mathcal{L}_{\text{CE}}^{(k)}, \qquad \lambda_k = 1 \text{ by default.} \tag{6}$$

The shared encoder thus receives gradients from all three tasks, while each head only sees its own data. The weights λ_k are changed only if one task clearly dominates training, and any change is reported.

Size budget. The weights must fit in the memory available on board, M bytes. If each weight takes β bytes, the deployed model (encoder plus the three heads) must satisfy

$$|\theta| + \sum_{k=1}^{3} |\psi_k| \le \frac{M}{\beta}, \tag{7}$$

where ψ_k are the weights of head h_k. Quantisation lowers β: each weight w is stored as an integer q with a scale s, $\hat{w} = s\,q$. INT8 ($\beta = 1$) fits four times more parameters than FP32 ($\beta = 4$), INT4 ($\beta = 0.5$) eight times more.

There are two ways to quantise a model. In post-training quantisation (PTQ) the model is trained normally and converted to integers at the end. In quantisation-aware training (QAT) the rounding is already applied during training, so the weights learn to work with it. PTQ is cheaper, QAT is more accurate when very few bits are used:

	PTQ	QAT
Extra training	none	yes
Accuracy at 8 bits	close to FP32	close to FP32
Accuracy at 4 bits	clear drop	mostly recovered

Table 1: Post-training vs quantisation-aware training.

The scale s can be one per layer (per-tensor) or one per output channel (per-channel). Per-channel costs a few extra bytes but is more accurate, because channels with small weights are not forced to share the scale of the largest ones.

TinyGFM uses INT8 PTQ with per-channel scales. At 8 bits PTQ is almost as accurate as QAT, needs no extra training, and INT8 is the format run by microcontroller libraries such as CMSIS-NN and TensorFlow Lite Micro. QAT is kept as a fallback, used only if the INT8 model loses too much accuracy or if INT4 is needed.

4 Student networks

5 Feature distillation

6 Multi-task fine-tuning

7 Quantisation and on-board constraints

8 Experimental design and evaluation

9 Results and analysis

10 Discussion, limitations and future work

Sources

Chapter 2: Dataset.
CloudSEN12: https://cloudsen12.github.io/; Aybar et al., *CloudSEN12, a global dataset for semantic understanding of cloud and cloud shadow in Sentinel-2*, Scientific Data 2022 (https://pmc.ncbi.nlm.nih.gov/articles/PMC9789947).
Sen1Floods11: https://github.com/cloudtostreet/Sen1Floods11; Bonafilia et al., CVPR Workshops 2020, pp. 210–211 (https://openaccess.thecvf.com/content_CVPRW_2020/papers/w11/Bonafilia_Sen1Floods11_A_Georeferenced_Dataset_to_Train_and_Test_Deep_Learning_CVPRW_2020_paper.pdf); split counts from arXiv:2302.08180.
THRawS: Zenodo https://zenodo.org/records/7908728 (DOI 10.5281/zenodo.7908728; code: ESA-PhiLab/PyRawS); Del Prete et al., *Unlocking the use of raw multispectral Earth observation imagery for onboard AI*, arXiv:2305.11891.
SSL4EO-S12 (unlabelled pool): Wang et al., *SSL4EO-S12: a large-scale multi-modal, multi-temporal dataset for self-supervised learning in Earth observation*, arXiv:2211.07044.

Chapter 3: Teacher models.
Prithvi-EO-2.0: Hugging Face https://huggingface.co/ibm-nasa-geospatial/Prithvi-EO-2.0-300M (weights and ViT-L configuration); Szwarcman et al., *Prithvi-EO-2.0: a versatile multi-temporal foundation model for Earth observation applications*, arXiv:2412.02732.
Copernicus-FM and Copernicus-Pretrain: Wang et al., *Towards a unified Copernicus foundation model for Earth vision*, arXiv:2503.11849.
DOFA (wavelength-conditioned patch embedding): Xiong et al., *Neural plasticity-inspired multimodal foundation model for Earth observation*, arXiv:2403.15356.
Vision Transformer: Dosovitskiy et al., *An image is worth 16x16 words: Transformers for image recognition at scale*, ICLR 2021, arXiv:2010.11929.
Masked autoencoders: He et al., *Masked autoencoders are scalable vision learners*, CVPR 2022, arXiv:2111.06377.
HLS archive: Claverie et al., *The Harmonized Landsat and Sentinel-2 surface reflectance data set*, Remote Sensing of Environment 219 (2018), pp. 145–161.
FLOP estimate: Kaplan et al., *Scaling laws for neural language models*, arXiv:2001.08361.
Quantisation, PTQ vs QAT (Table 1): Nagel et al., *A white paper on neural network quantization*, 2021, arXiv:2106.08295 https://arxiv.org/abs/2106.08295; Krishnamoorthi, *Quantizing deep convolutional networks for efficient inference: a whitepaper*, 2018, arXiv:1806.08342 https://arxiv.org/abs/1806.08342.
Integer-only inference: Jacob et al., *Quantization and training of neural networks for efficient integer-arithmetic-only inference*, CVPR 2018, arXiv:1712.05877.
Microcontroller libraries: Lai et al., *CMSIS-NN: efficient neural network kernels for Arm Cortex-M CPUs*, arXiv:1801.06601; David et al., *TensorFlow Lite Micro: embedded machine learning on TinyML systems*, MLSys 2021, arXiv:2010.08678.